

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)
Факультет компьютерных технологий и прикладной математики
Кафедра информационных технологий

ОТЧЕТ
о выполнении лабораторной работы №5
по дисциплине «Системы реального времени»

Работу выполнил студент 42/2 группы

Д.А. Вербицкий

Направление подготовки 02.03.03 Математическое обеспечение и
администрирование информационных систем курс 4

Проверил доцент кафедры информационных технологий

А.Н. Полетайкин

Краснодар

2025

Вариант 8

Тема: организация обмена между датчиками, УВМ и исполнительными устройствами.

Цель: изучение принципов организации инфообмена между ядром СРВ и периферийными устройствами, приобретение практических навыков разработки функциональных схем, алгоритмов и управляющих программ для управления процессами в режиме реального времени.

Задание

Задан технологический процесс или функционирование объекта управления, контроль за состоянием которого осуществляется с помощью аналоговых и дискретных датчиков. Для аналоговых датчиков определены допустимый диапазон измерений, диапазон выходного сигнала, инерционность и интервал или режим опроса. Известен АЦП, преобразующий входную аналоговую информацию в двоичный код. В соответствии с измеряемыми значениями необходимо выдать аналоговое управляющее воздействие на исполнительные устройства, регулирующие протекание технологического процесса или состояние объекта управления. Для вывода аналоговой информации используется ЦАП. Разрядности АЦП и ЦАП, диапазоны аналоговых сигналов и максимальное время преобразования ЦАП заданы в табл. 1.

№ вар.	Разрядность АЦП	Разрядность ЦАП	Диапазон напряжения, В		Максимальное время преобразования ЦАП, мс
			Входной сигнал	Выходное воздействие	
8	12	10	-10 ... 10	-6,4...12,8	80

Таблица 1 – Вариант 8

Индивидуальные условия

Имеется конвейер, по которому движутся детали. Необходимо производить контроль размеров деталей. Размер детали определяется двумя дискретными датчиками. Первый выдает 1, если размер меньше нормального. Второй выдает 0, если размер больше нормального. В первом случае открывается заслонка корзины бракованных деталей посредством электромагнитного привода. Во втором случае включается поворотное устройство, изменяющее направление движения детали в цех обработки. Поворотное устройство управляется двигателем, на который необходимо подать напряжение 11,3 В в течение 36 с. Для установки в исходное положение на двигатель поворотного устройства необходимо подать напряжение –2,2 В. Следует предусмотреть время для отработки команд. У деталей, направляемых в цех обработки, производится измерение температуры с помощью 2 бесконтактных датчиков, измеряющих температуру в диапазоне – 30°C - 95°C, диапазон выходного сигнала –3 - 9,5В. Интервалы опроса соответственно равны 0,1 и 0,5 с. Датчики опрашиваются до тех пор, пока среднее значение измеряемой температуры не достигнет 60 °C.

Порядок выполнения работы

1. Проанализировать индивидуальное задание и сформировать список компонентов УСО СРВ с указанием их основных характеристик.
2. Определить количество, разрядность и назначение портов ввода-вывода, а также назначение отдельных разрядов портов. Использовать табличную форму представления.
3. Разработать структурную схему интерфейса PDI.
4. Построить градуировочную характеристику для каждого аналогового датчика (при наличии).
5. Рассчитать двоичные эквиваленты контрольных и управляющих непрерывных величин.
6. Разработать алгоритм функционирования объекта.

7. Рассчитать параметры задержки процедур(ы) временной.
8. Составить программу управления (драйвер) на языке ассемблера.
9. На языке высокого уровня разработать приложение, в графическом режиме имитирующее функционирование СРВ. Масштаб времени может быть любым.

Ход работы

Пункт 1

Дискретный_Датчик_1

Определяет размер детали. Выдаёт 1, если размер меньше нормального.

Дискретный_Датчик_2

Определяет размер детали. Выдаёт 0, если размер больше нормального.

Поворотное_Устройство

Изменяет направление движения детали в цех обработки. Управляется двигателем, на который необходимо подать напряжение 11,3 В в течение 36 с. Для установки в исходное положение на двигатель поворотного устройства необходимо подать напряжение $-2,2$ В.

Заслонка_Корзины_Бракованных_Деталей

Открывается посредством электромагнитного привода, если первый датчик выдал 1.

Бесконтактный_Датчик_1

Измеряет температуру в диапазоне -30°C - 95°C , диапазон выходного сигнала -3 - $9,5\text{В}$. Опрашивается до тех пор, пока среднее значение измеряемой температуры не достигнет 60°C .

Бесконтактный_Датчик_2

Измеряет температуру в диапазоне -30°C - 95°C , диапазон выходного сигнала -3 - $9,5\text{В}$. Опрашивается до тех пор, пока среднее значение измеряемой температуры не достигнет 60°C .

Пункт 2

Адрес порта	Общее назначение	Разрядность	Разряд	Назначение разряда
300h	передача данных с АЦП на УВМ	16	0-11	Данные с АЦП 12-битный код (0-4095)
			12	ДР1 (дискретный датчик размера 1): 1 — размер меньше нормы, 0 — иначе
			13	ДР2 (дискретный датчик размера 2, инверсный): 0 — размер больше нормы, 1 — иначе
			14	Не используется
			15	ГТ (Готовность АЦП) 1 = данные готовы 0 = преобразование не завершено
301h	1)передача данных с УВМ на ЦАП 2) выбор датчика температуры 3) запуск ЦАП 4) запуск АЦП	16	0-9	Данные для ЦАП 10-битный код (0-1023)
			10	Выбор датчика температуры: 0 = Бесконтактный датчик 1 1 = Бесконтактный датчик 2
			11	Упр. заслонкой брака: 1 — открыть/активировать,

				0 — выключить
			12	Команда “Поворот в цех обработки”: 1 — выполнить, 0 — нет
			13	Команда “Возврат в исходное положение”: 1 — выполнить, 0 — нет
			14	Запуск ЦАП: 1 = данные готовы к преобразованию ЦАП, запуск (обновление выходного воздействия) 0 = запускать ЦАП не нужно ($t_{\text{преобр}} \leq 80 \text{ мс}$)
			15	Запуск АЦП: 1 = старт преобразования АЦП 0 = не запускать АЦП

Пункт 3

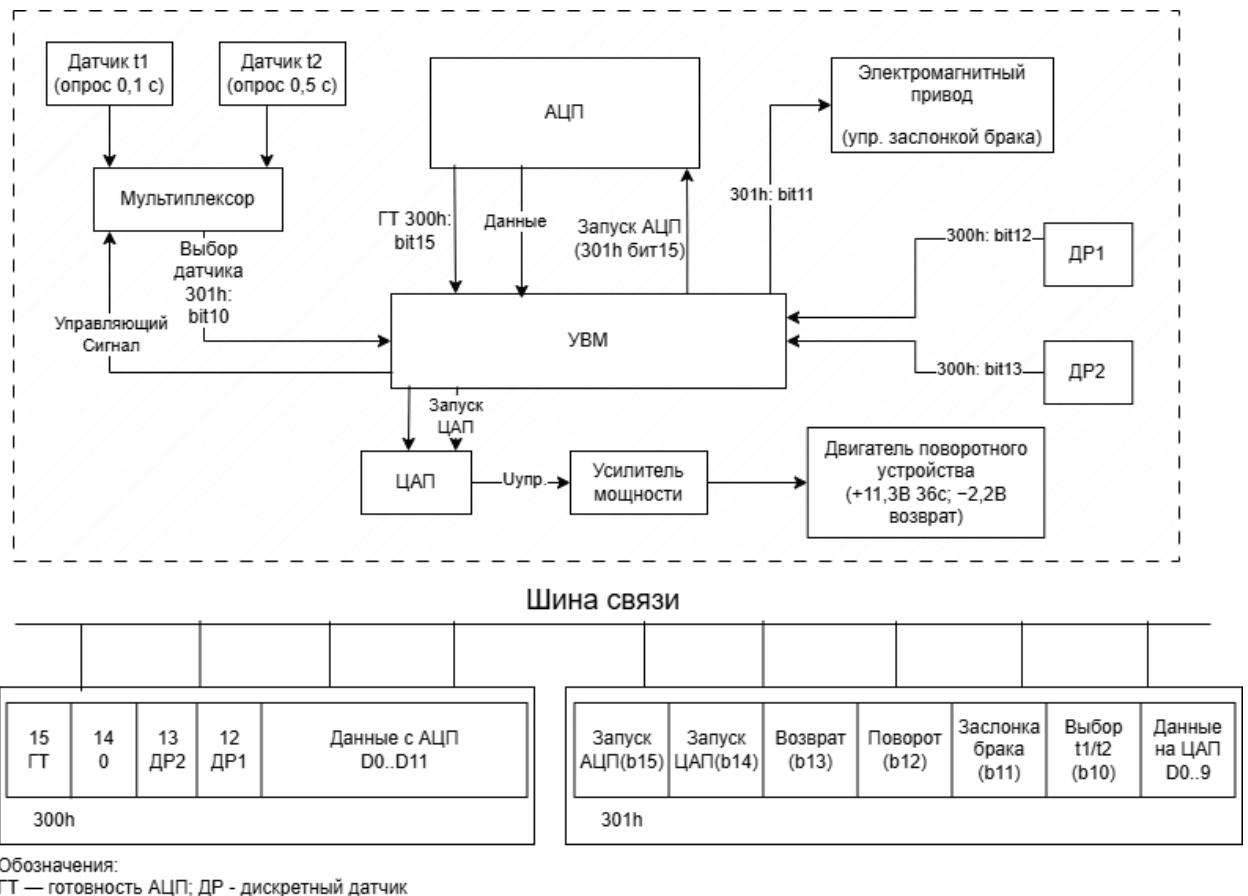


Рисунок 1 – Структурная схема СРВ

Пункт 4

Измерение температуры осуществляется двумя бесконтактными датчиками, подключенными через мультиплексор к АЦП. Датчики измеряют температуру в диапазоне $T = -30 \dots 95 \text{ }^{\circ}\text{C}$ и формируют выходное напряжение в диапазоне $U = -3 \dots 9,5 \text{ В}$. Интервалы опроса датчиков составляют 0,1 с и 0,5 с. Опрос выполняется до тех пор, пока среднее значение температуры не достигнет $60 \text{ }^{\circ}\text{C}$.

Примем градуировочную характеристику датчиков линейной и зададим ее по крайним точкам диапазона:

$$U(T) = k \cdot (T - T_{\min}) + U_{\min}$$

где T - температура, $^{\circ}\text{C}$; U - выходное напряжение датчика, В; $T_{\min} = -30 \text{ }^{\circ}\text{C}$, $T_{\max} = 95 \text{ }^{\circ}\text{C}$; $U_{\min} = -3 \text{ В}$, $U_{\max} = 9,5 \text{ В}$.

Коэффициент преобразования:

$$k = (U_{\max} - U_{\min}) / (T_{\max} - T_{\min}) = (9,5 - (-3)) / (95 - (-30)) = 12,5 / 125 = 0,1 \text{ В/}^{\circ}\text{C}$$

Тогда:

$$U(T) = 0,1 \cdot (T + 30) - 3$$

После упрощения получаем:

$$U(T) = 0,1 \cdot T$$

Проверка: при $T = -30^{\circ}\text{C}$, $U = -3 \text{ В}$; при $T = 95^{\circ}\text{C}$, $U = 9,5 \text{ В}$. Таким образом, характеристика соответствует заданным пределам.

Для построения графика градуировочной характеристики выберем несколько характерных точек температурного диапазона. Так как $U(T) = 0,1 \cdot T$, значения напряжения определяются прямой пропорциональностью.

T, °C	-30	0	30	60	95
U(T), В	-3,0	0,0	3,0	6,0	9,5

По данным таблицы строится график: по оси X откладывается температура T, по оси Y - напряжение U. График представляет собой прямую линию на участке $T \in [-30; 95]$.

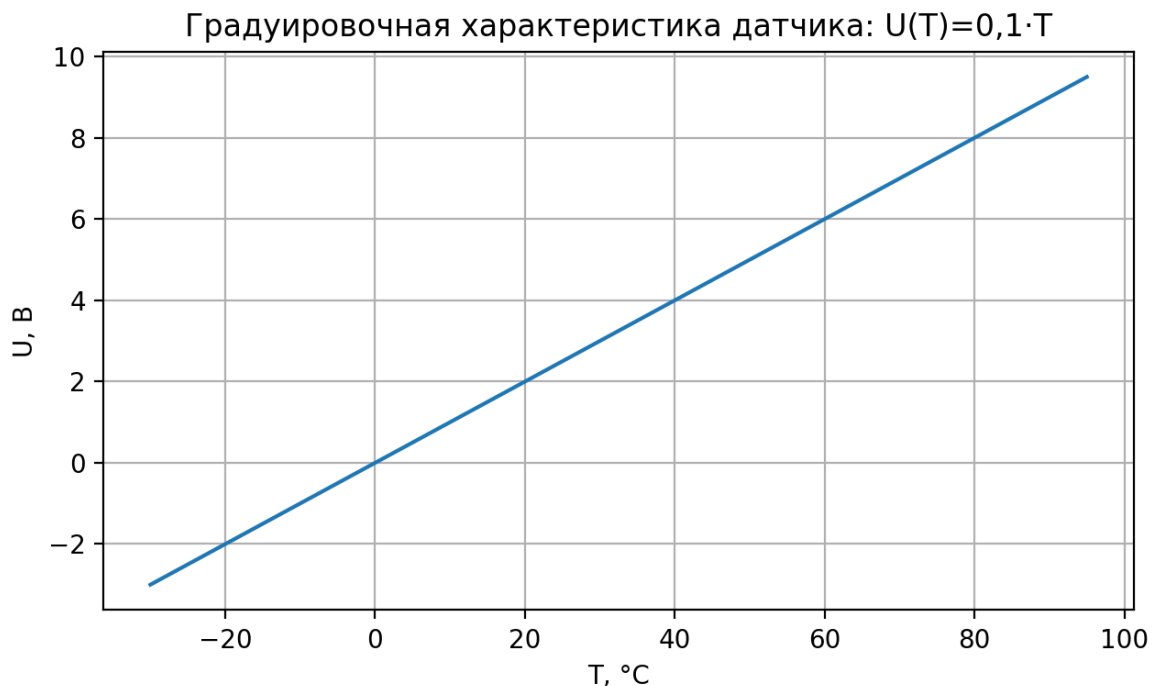


Рисунок 2 – Градуировочная характеристика датчика температуры

Выбор активного датчика осуществляется мультиплексором (бит 10 порта 301h). Значение напряжения датчика преобразуется АЦП ($n = 12$, диапазон $-10...+10$ В), после чего в порт 300h выдаются данные D0...D11 и сигнал готовности ГТ (бит 15). Полученный цифровой код далее используется в пункте 5 для расчета цифровых эквивалентов.

Для контрольной температуры $60\text{ }^{\circ}\text{C}$ по характеристике получаем:

$$U(60) = 0,1 \cdot 60 = 6,0 \text{ В}$$

Пункт 5

Используются следующие параметры преобразователей: АЦП: $n = 12$ бит, диапазон входного напряжения $U_{\text{вх}} = -10...+10$ В; ЦАП: $m = 10$ бит, диапазон выходного напряжения $U_{\text{вых}} = -6,4...+12,8$ В, максимальное время преобразования $t_{\text{пр}} \leq 80$ мс.

Выходное напряжение датчика температуры (п.4) описывается линейной характеристикой $U(T) = 0,1 \cdot T$. Цифровой код АЦП вычисляется по формуле:

$$D_ADC = \text{ROUND}((U - U_{\text{min}})/(U_{\text{max}} - U_{\text{min}}) \cdot 2^n), \text{ где } U_{\text{min}} = -10 \text{ В, } U_{\text{max}} = +10 \text{ В, } n = 12.$$

Рассчитаем двоичный эквивалент (код АЦП) для нескольких контрольных точек температурного диапазона $T = -30...95\text{ }^{\circ}\text{C}$ (для датчиков t_1 и t_2 значения одинаковы, различается только интервал опроса).

T, $^{\circ}\text{C}$	U(T), В	D_ADC (10)	D_ADC (16)
-30	-3.0	1434	059Ah
0	0.0	2048	0800h
30	3.0	2662	0A66h
60	6.0	3277	0CCDh
95	9.5	3994	0F9Ah

Контрольное значение для сравнения при достижении средней температуры $60\text{ }^{\circ}\text{C}$: $D_{\text{конт}} = 0\text{CCDh}$.

Управление исполнительным механизмом осуществляется двумя уровнями напряжения: $U_{\text{упр1}} = +11,3$ В (поворот) и $U_{\text{упр2}} = -2,2$ В (возврат). Код ЦАП вычисляется по формуле:

$$D_DAC = \text{ROUND}((U - U_{\text{min}})/(U_{\text{max}} - U_{\text{min}}) \cdot 2^m), \text{ где } U_{\text{min}} = -6,4 \text{ В, } U_{\text{max}} = +12,8 \text{ В, } m = 10.$$

Рассчитаем коды для указанных управляющих воздействий:

- Для $U_{упр} = +11,3 \text{ В}$: $D1 = 94410 = 3B0h$.
- Для $U_{упр} = -2,2 \text{ В}$: $D2 = 22410 = 0E0h$.

Полученные 10-разрядные коды заносятся в разряды $D0...D9$ порта $301h$, после чего устанавливается бит 14 (запуск ЦАП). При формировании управляющего воздействия следует учитывать $t_{пр} \leq 80 \text{ мс}$ (обновление выходного напряжения ЦАП не чаще указанного времени).

Пункт 6

Дискретные датчики размера формируют сигналы $DR1$ и $DR2$ и подают их на входной порт $300h$: $DR1$ — bit12, $DR2$ — bit13 (инверсный).

Бесконтактные датчики температуры $t1$ и $t2$ формируют аналоговый сигнал $U(T)$ в диапазоне $-3...9,5 \text{ В}$; выбор активного датчика выполняется мультиплексором по команде с порта $301h$ (bit10).

АЦП ($n=12$, $U_{вх}=-10...+10 \text{ В}$) по сигналу «Запуск АЦП» ($301h$ bit15) выполняет преобразование и записывает 12-разрядный код в разряды $D0...D11$ порта $300h$, а также устанавливает признак готовности ГТ в bit15 порта $300h$.

ЦАП ($m=10$, $U_{вых}=-6,4...+12,8 \text{ В}$) получает код из разрядов $D0...D9$ порта $301h$ и по сигналу «Запуск ЦАП» ($301h$ bit14) формирует управляющее напряжение на двигатель через усилитель мощности ($t_{пр} \leq 80 \text{ мс}$).

Дискретные команды исполнительным устройствам выдаются из порта $301h$: bit11 — электромагнит заслонки брака; bit12 — команда «Поворот»; bit13 — команда «Возврат».

Алгоритм функционирования объекта (управляющая программа УВМ):

1. Считать слово порта $300h$ и выделить дискретные признаки $DR1$ (bit12) и $DR2$ (bit13).
2. Если $DR1=1$ (размер меньше нормы), то установить в порте $301h$ бит 11 (заслонка брака) =1; остальные управляющие биты (12,13) сбросить. Деталь направляется в корзину брака, дальнейшие действия не выполняются.
3. Если $DR2=0$ (размер больше нормы, инверсный датчик), то направить деталь в цех обработки: выдать на ЦАП управляющее напряжение $+11,3 \text{ В}$ (код $D1=3B0h$ в разряды $D0...D9$ порта $301h$), установить bit12=1 (команда

«Поворот») и bit14=1 (запуск ЦАП). Выдержать паузу не менее $t_{pr}(ЦАП)=80$ мс, затем удерживать воздействие в течение 36 с.

4. Если DR1=0 (размер детали не меньше нормы) и DR2=1 (размер детали не больше нормы, поскольку датчик DR2 инверсный), то деталь считается соответствующей норме. В этом случае бракование не выполняется: заслонка брака должна быть отключена (в порте 301h бит 11 = 0), команды исполнительному механизму не формируются (в порте 301h биты 12 «Поворот» и 13 «Возврат» устанавливаются в 0), а обновление управляющего напряжения через ЦАП не требуется (бит 14 «Запуск ЦАП» = 0). Деталь пропускается дальше по конвейеру без направления в корзину брака и без отправки в цех обработки, после чего управляющая программа переходит к обработке следующей детали (к следующему циклу опроса DR1/DR2).

5. После завершения поворота выполнить возврат механизма в исходное положение: выдать на ЦАП напряжение $-2,2$ В (код D2=0E0h), установить bit13=1 (команда «Возврат») и bit14=1 (запуск ЦАП). Выдержать паузу ≥ 80 мс и обеспечить время отработки возврата (в отчёте принимается равным времени поворота, если иное не задано экспериментально).

6. После направления детали в цех обработки выполнить цикл измерения температуры двумя датчиками до достижения среднего значения $60\text{ }^{\circ}\text{C}$:

а) выбрать датчик t1 (301h bit10=0), подать сигнал «Запуск АЦП» (301h bit15=1), ожидать $GT=1$ (300h bit15), считать код D0...D11, преобразовать в U и T1;

б) с периодом 0,5 с аналогично опрашивать датчик t2 (301h bit10=1) и получить T2;

в) вычислять среднее значение $T_{cp}=(T1+T2)/2$ и сравнивать с порогом $60\text{ }^{\circ}\text{C}$. При $T_{cp} \geq 60\text{ }^{\circ}\text{C}$ завершить цикл контроля температуры.

7. Сбросить управляющие биты порта 301h (11–15) в исходное состояние и перейти к обработке следующей детали (переход к шагу 1).

Пункт 7

Параметры временных задержек:

Интервал опроса датчика t1: $\Delta t1 = 0,1$ с.

Интервал опроса датчика t2: $\Delta t2 = 0,5$ с.

Время воздействия на двигатель при повороте: $t_{\text{повор}} = 36 \text{ с}$

Время преобразования ЦАП: $t_{\text{пр}}(\text{ЦАП}) \leq 80 \text{ мс}$; после установки кода и сигнала запуска необходимо выдерживать паузу не менее 80 мс до следующего обновления.

Время преобразования АЦП не задано; для расчётов принимается типовое значение $t_{\text{пр}}(\text{АЦП}) \approx 20 \text{ мкс}$ (влияние на интервалы 0,1/0,5с пренебрежимо мало).

Время установления после переключения мультиплексора принимается туст $\approx 20 \text{ мс}$ (пауза перед запуском АЦП при смене датчика).

В управляющей программе задержки реализуются программно (подпрограммами Delay0_1s, Delay0_5s, Delay80ms, Delay36s).

Пункт 8

.686

.model flat, stdcall

option casemap:none

include \masm32\include\windows.inc

include \masm32\include\kernel32.inc

include \masm32\include\user32.inc

includelib \masm32\lib\kernel32.lib

includelib \masm32\lib\user32.lib

PORT300 equ 0300h

PORT301 equ 0301h

MASK_ADC equ 0FFFh

BIT_GT equ 8000h

BIT_DR1 equ 1000h

BIT_DR2 equ 2000h

BIT_SA equ 8000h

BIT_SD equ 4000h

BIT_RET equ 2000h

BIT_TURN equ 1000h

BIT_EM equ 0800h

BIT_SEL equ 0400h

MASK_DAC equ 03FFh

CODE_TURN equ 03B0h

CODE_RET equ 00E0h

TH_T60 equ 0CCDh

.data

ctrl301 dw 0

t2_last dw 0

cnt5 db 0

.code

ReadADC_MUX proc uses edx ebx

mov bx, ctrl301

test al, al

jz @sel_t1

or bx, BIT_SEL

jmp @sel_done

@sel_t1:

and bx, NOT BIT_SEL

@sel_done:

mov ctrl301, bx

mov dx, PORT301

mov ax, bx

out dx, ax

call Delay20ms

or bx, BIT_SA

mov ctrl301, bx

mov ax, bx

out dx, ax

```
and bx, NOT BIT_SA
mov ctrl301, bx
mov ax, bx
out dx, ax
```

@wait_gt:

```
mov dx, PORT300
in ax, dx
test ax, BIT_GT
jz @wait_gt
```

```
and ax, MASK_ADC
ret
```

ReadADC_MUX endp

WriteDAC_Cmd proc uses edx ebx

```
mov bx, ctrl301
and bx, (BIT_SEL or BIT_EM)
```

```
and ax, MASK_DAC
or bx, ax
or bx, cx
```

```
or bx, BIT_SD
mov ctrl301, bx
mov dx, PORT301
mov ax, bx
out dx, ax
call Delay80ms
```

```
and bx, NOT BIT_SD
mov ctrl301, bx
mov ax, bx
out dx, ax
ret
```

WriteDAC_Cmd endp

Delay20ms proc uses ecx edx

mov ecx, 0500h

@o1: mov edx, 0FFFFh

@i1: dec edx

jne @i1

loop @o1

ret

Delay20ms endp

Delay80ms proc uses ecx

mov ecx, 4

@l80:

call Delay20ms

loop @l80

ret

Delay80ms endp

Delay100ms proc uses ecx

mov ecx, 5

@l100:

call Delay20ms

loop @l100

ret

Delay100ms endp

Delay1s proc uses ecx

mov ecx, 10

@l1s:

call Delay100ms

loop @l1s

ret

Delay1s endp

Delay36s proc uses ecx

mov ecx, 36

@l36:

call Delay1s

loop @l36

ret

Delay36s endp

start:

xor ax, ax

mov ctrl301, ax

mov dx, PORT301

out dx, ax

main_loop:

mov dx, PORT300

in ax, dx

test ax, BIT_DR1

jnz do_reject

test ax, BIT_DR2

jz do_oversize

jmp main_loop

do_reject:

mov bx, ctrl301

or bx, BIT_EM

mov ctrl301, bx

mov dx, PORT301

mov ax, bx

out dx, ax

call Delay80ms

and bx, NOT BIT_EM

```
mov ctrl301, bx
mov ax, bx
out dx, ax
jmp main_loop
```

do_oversize:

```
mov ax, CODE_TURN
mov cx, BIT_TURN
call WriteDAC_Cmd
call Delay36s
```

```
mov bx, ctrl301
and bx, NOT BIT_TURN
mov ctrl301, bx
mov dx, PORT301
mov ax, bx
out dx, ax
```

```
mov ax, CODE_RET
mov cx, BIT_RET
call WriteDAC_Cmd
call Delay36s
```

```
mov bx, ctrl301
and bx, NOT BIT_RET
mov ctrl301, bx
mov dx, PORT301
mov ax, bx
out dx, ax
```

```
mov cnt5, 0
mov t2_last, 0
```

temp_loop:

```
xor al, al
```

```

call ReadADC_MUX
mov bx, ax

mov al, cnt5
inc al
mov cnt5, al
cmp al, 5
jb @skip_t2

mov cnt5, 0
mov al, 1
call ReadADC_MUX
mov t2_last, ax

@skip_t2:
mov ax, t2_last
add ax, bx
shr ax, 1

cmp ax, TH_T60
jae main_loop

call Delay100ms
jmp temp_loop

end_program:
    invoke ExitProcess, 0

end start

```

Пункт 9

```

import tkinter as tk
from tkinter import ttk
import time
import threading
from dataclasses import dataclass

```

```

ADC_N = 12
ADC_U1 = -10.0
ADC_U2 = 10.0

DAC_M = 10
DAC_U1 = -6.4
DAC_U2 = 12.8

PORT300 = 0x300
PORT301 = 0x301

MASK_ADC = (1 << ADC_N) - 1
MASK_DAC = (1 << DAC_M) - 1

BIT_GT = 15
BIT_DR1 = 12
BIT_DR2 = 13

BIT_SEL = 10
BIT_EM = 11
BIT_TURN = 12
BIT_RET = 13
BIT_SD = 14
BIT_SA = 15

MUX_SWITCH = 0.02
DAC_CONV = 0.08
EM_PULSE = 0.08
T1_PERIOD = 0.1
T2_PERIOD = 0.5

TURN_TIME = 36.0
RET_TIME = 36.0

CODE_TURN = 0x3B0
CODE_RET = 0x0E0
TH_T60 = 0x0CCD

def clamp(x, a, b):
    return max(a, min(b, x))

def bit(val: int, n: int) -> int:
    return (val >> n) & 1

def set_bit(val: int, n: int, b: int) -> int:
    if b:
        return val | (1 << n)
    return val & ~(1 << n)

def adc_u_from_temp(t_c: float) -> float:
    return 0.1 * t_c

```

```

def adc_code_from_temp(t_c: float) -> int:
    u = adc_u_from_temp(t_c)
    u = clamp(u, ADC_U1, ADC_U2)
    levels = 2 ** ADC_N
    code = int(round((u - ADC_U1) / (ADC_U2 - ADC_U1) * levels))
    return int(clamp(code, 0, levels - 1))

def dac_u_from_code(code: int) -> float:
    levels = 2 ** DAC_M
    code = int(clamp(code, 0, levels - 1))
    return DAC_U1 + (DAC_U2 - DAC_U1) * (code / levels)

@dataclass
class Ports:
    port300: int = 0
    port301: int = 0

class Variant8Sim:
    def __init__(self, root: tk.Tk):
        self.root = root
        self.root.title("JIP5 CPB – Вариант 8 (Пункт 9: имитационная модель)")

        self.ports = Ports()
        self.lock = threading.Lock()
        self.running = True

        self.time_scale = tk.DoubleVar(value=10.0)
        self.auto_heat = tk.BooleanVar(value=False)

        self.t1 = tk.DoubleVar(value=25.0)
        self.t2 = tk.DoubleVar(value=25.0)

        self.size_mode = tk.StringVar(value="normal")
        self.status = tk.StringVar(value="Ожидание...")

        self.thread = None

        self._ui()

        self.apply_size_bits()
        self.ui_update()

        self.root.protocol("WM_DELETE_WINDOW", self.on_close)

    def _ui(self):
        main = ttk.Frame(self.root, padding=12)
        main.grid(row=0, column=0, sticky="nsew")
        self.root.columnconfigure(0, weight=1)
        self.root.rowconfigure(0, weight=1)
        main.columnconfigure(0, weight=1)
        main.columnconfigure(1, weight=1)

```

```

        lf_s = ttk.LabelFrame(main, text="Аналоговые датчики температуры",
padding=10)
        lf_s.grid(row=1, column=0, sticky="nsew", padx=(0, 8))

        ttk.Label(lf_s, text="t1 (оппос 0.1c)").grid(row=0, column=0,
sticky="w")
        ttk.Scale(lf_s, from_=-40, to=240, variable=self.t1,
orient="horizontal").grid(row=1, column=0, sticky="ew")
        self.lbl_t1 = ttk.Label(lf_s, text="")
        self.lbl_t1.grid(row=2, column=0, sticky="w", pady=(0, 8))

        ttk.Label(lf_s, text="t2 (оппос 0.5c)").grid(row=3, column=0,
sticky="w")
        ttk.Scale(lf_s, from_=-40, to=240, variable=self.t2,
orient="horizontal").grid(row=4, column=0, sticky="ew")
        self.lbl_t2 = ttk.Label(lf_s, text="")
        self.lbl_t2.grid(row=5, column=0, sticky="w")

        ttk.Checkbutton(lf_s, text="Автоизменение t1/t2 в цикле (демо)",
variable=self.auto_heat)\
            .grid(row=6, column=0, sticky="w", pady=(8, 0))

        lf_s.columnconfigure(0, weight=1)

        lf_d = ttk.LabelFrame(main, text="Дискретные датчики размера (сидят в
300h)", padding=10)
        lf_d.grid(row=2, column=0, sticky="nsew", padx=(0, 8), pady=(8, 0))

        ttk.Radiobutton(lf_d, text="Норма: DR1=0, DR2=1", value="normal",
variable=self.size_mode,
                        command=self.apply_size_bits).grid(row=0, column=0,
sticky="w")
        ttk.Radiobutton(lf_d, text="Меньше нормы: DR1=1", value="undersize",
variable=self.size_mode,
                        command=self.apply_size_bits).grid(row=1, column=0,
sticky="w")
        ttk.Radiobutton(lf_d, text="Больше нормы: DR2 инверсный → 0",
value="oversize", variable=self.size_mode,
                        command=self.apply_size_bits).grid(row=2, column=0,
sticky="w")

        ttk.Button(lf_d, text="Запустить цикл (как в драйвере)",
command=self.start_cycle)\
            .grid(row=3, column=0, sticky="ew", pady=(8, 0))
        lf_d.columnconfigure(0, weight=1)

        lf_p = ttk.LabelFrame(main, text="Порты 300h/301h и индикация",
padding=10)
        lf_p.grid(row=1, column=1, rowspan=2, sticky="nsew")

        self.lbl_300 = ttk.Label(lf_p, text="", font=("Consolas", 10))
        self.lbl_300.grid(row=0, column=0, sticky="w")

        self.lbl_301 = ttk.Label(lf_p, text="", font=("Consolas", 10))
        self.lbl_301.grid(row=1, column=0, sticky="w", pady=(6, 10))

```

```

self.ind_sel = ttk.Label(lf_p, text="SEL: 0 (t1)")
self.ind_sel.grid(row=2, column=0, sticky="w")
self.ind_gt = ttk.Label(lf_p, text="GT: 0")
self.ind_gt.grid(row=3, column=0, sticky="w")

self.ind_em = ttk.Label(lf_p, text="ЕМ (заслонка): OFF")
self.ind_em.grid(row=4, column=0, sticky="w", pady=(6, 0))

self.ind_turn = ttk.Label(lf_p, text="TURN (поворот): OFF")
self.ind_turn.grid(row=5, column=0, sticky="w")

self.ind_ret = ttk.Label(lf_p, text="RET (возврат): OFF")
self.ind_ret.grid(row=6, column=0, sticky="w")

self.ind_dac = ttk.Label(lf_p, text="DAC: code=000h, U=0.00V")
self.ind_dac.grid(row=7, column=0, sticky="w", pady=(6, 0))

ttk.Label(lf_p, text="Масштаб времени (больше =
быстрее):").grid(row=8, column=0, sticky="w", pady=(10, 0))
ttk.Scale(lf_p, from_=1, to=50, variable=self.time_scale,
orient="horizontal").grid(row=9, column=0, sticky="ew")
lf_p.columnconfigure(0, weight=1)

lf_st = ttk.LabelFrame(main, text="Статус", padding=10)
lf_st.grid(row=3, column=0, columnspan=2, sticky="nsew", pady=(10,
0))

ttk.Label(lf_st, textvariable=self.status, font=("Arial",
11)).grid(row=0, column=0, sticky="w")

def _sleep(self, sec: float):
    scale = max(0.1, float(self.time_scale.get()))
    time.sleep(sec / scale)

def apply_size_bits(self):
    with self.lock:
        p = self.ports.port300
        p = set_bit(p, BIT_DR1, 0)
        p = set_bit(p, BIT_DR2, 0)

        if self.size_mode.get() == "undersize":
            p = set_bit(p, BIT_DR1, 1)
            p = set_bit(p, BIT_DR2, 1)
        elif self.size_mode.get() == "oversize":
            p = set_bit(p, BIT_DR1, 0)
            p = set_bit(p, BIT_DR2, 0)
        else:
            p = set_bit(p, BIT_DR1, 0)
            p = set_bit(p, BIT_DR2, 1)

    self.ports.port300 = p

def _write_301(self, new301: int):
    with self.lock:
        self.ports.port301 = new301 & 0xFFFF

```

```

def _read_300(self) -> int:
    with self.lock:
        return self.ports.port300 & 0xFFFF

def _set_300(self, new300: int):
    with self.lock:
        self.ports.port300 = new300 & 0xFFFF

def _set_gt(self, v: int):
    with self.lock:
        p = self.ports.port300
        p = set_bit(p, BIT_GT, 1 if v else 0)
        self.ports.port300 = p

def _set_301_bit(self, n: int, v: int):
    with self.lock:
        p = self.ports.port301
        p = set_bit(p, n, 1 if v else 0)
        self.ports.port301 = p

def _set_301_dac(self, code: int):
    code &= MASK_DAC
    with self.lock:
        p = self.ports.port301
        p = (p & ~MASK_DAC) | code
        self.ports.port301 = p

def read_adc_mux(self, sel: int) -> int:
    self._set_301_bit(BIT_SEL, 1 if sel else 0)
    self._sleep(MUX_SWITCH)

    self._set_301_bit(BIT_SA, 1)
    self._sleep(0.001)
    self._set_301_bit(BIT_SA, 0)

    t = float(self.t2.get()) if sel else float(self.t1.get())
    code = adc_code_from_temp(t) & MASK_ADC

    p300 = self._read_300()
    p300 = (p300 & ~MASK_ADC) | code
    self._set_300(p300)

    self._set_gt(1)
    self._sleep(0.02)
    self._set_gt(0)

    return code

def write_dac_cmd(self, code: int, turn: int, ret: int, em: int):
    with self.lock:
        p = self.ports.port301
        sel = bit(p, BIT_SEL)
        p = 0
        p = set_bit(p, BIT_SEL, sel)
        p = set_bit(p, BIT_EM, 1 if em else 0)
        p = set_bit(p, BIT_TURN, 1 if turn else 0)

```



```

        p = set_bit(p, BIT_RET, 1 if ret else 0)
        p = (p & ~MASK_DAC) | (code & MASK_DAC)
        self.ports.port301 = p

        self._set_301_bit(BIT_SD, 1)
        self._sleep(DAC_CONV)
        self._set_301_bit(BIT_SD, 0)

    def pulse_em(self):
        self.write_dac_cmd(self.ports.port301 & MASK_DAC, 0, 0, 1)
        self._sleep(EM_PULSE)
        self.write_dac_cmd(self.ports.port301 & MASK_DAC, 0, 0, 0)

    def start_cycle(self):
        if self.thread and self.thread.is_alive():
            return
        self.thread = threading.Thread(target=self._cycle, daemon=True)
        self.thread.start()

    def _cycle(self):
        p300 = self._read_300()
        dr1 = bit(p300, BIT_DR1)
        dr2 = bit(p300, BIT_DR2)

        if dr1 == 1:
            self.status.set("DR1=1 → меньше нормы → брак → импульс  
электромагнита заслонки (80мс)")
            self.pulse_em()
            self.status.set("Готово: брак отброшен, ожидание.")
            return

        if dr2 == 0:
            self.status.set("DR2=0 (инверсный) → больше нормы → поворот 36с  
(+11.3В) → возврат 36с (-2.2В)")
            self.write_dac_cmd(CODE_TURN, 1, 0, 0)
            self._sleep(TURN_TIME)
            self.write_dac_cmd(CODE_TURN, 0, 0, 0)

            self.write_dac_cmd(CODE_RET, 0, 1, 0)
            self._sleep(RET_TIME)
            self.write_dac_cmd(CODE_RET, 0, 0, 0)

            cnt = 0
            t2_last = 0

            self.status.set("Температурный цикл: t1 каждые 0.1с, t2 раз в  
0.5с, сравнение среднего кода с TH_T60")

            while True:
                if self.auto_heat.get():
                    self.t1.set(clamp(self.t1.get() + 0.6, -40, 240))
                    self.t2.set(clamp(self.t2.get() + 0.2, -40, 240))

                t1_code = self.read_adc_mux(0)

                if cnt % 5 == 0:

```

```

        t2_last = self.read_adc_mux(1)

        avg = (t1_code + t2_last) // 2
        self.status.set(f"t1_code=0x{t1_code:03X},
t2_code=0x{t2_last:03X}, avg=0x{avg:03X} ; порог TH=0x{TH_T60:03X}")

        if avg >= TH_T60:
            self.status.set("Условие выполнено: avg_code >= TH_T60.
Цикл завершён.")
            break

        cnt += 1
        self._sleep(T1_PERIOD)

    return

    self.status.set("Норма: DR1=0 и DR2=1 → управляющих воздействий
нет.")

    def ui_update(self):
        if not self.running:
            return

        with self.lock:
            p300 = self.ports.port300 & 0xFFFF
            p301 = self.ports.port301 & 0xFFFF

            t1 = float(self.t1.get())
            t2 = float(self.t2.get())
            c1 = adc_code_from_temp(t1)
            c2 = adc_code_from_temp(t2)

            self.lbl_t1.config(text=f"t1={t1:.1f}°C →
U={adc_u_from_temp(t1):.2f}V → АЦП≈0x{c1:03X}")
            self.lbl_t2.config(text=f"t2={t2:.1f}°C →
U={adc_u_from_temp(t2):.2f}V → АЦП≈0x{c2:03X}")

            self.lbl_300.config(text=f"300h=0x{p300:04X}  GT={bit(p300,BIT_GT)}
DR2(inv)={bit(p300,BIT_DR2)} DR1={bit(p300,BIT_DR1)}  ADC=0x{(p300 &
MASK_ADC):03X}")
            self.lbl_301.config(text=f"301h=0x{p301:04X}  SA={bit(p301,BIT_SA)}
SD={bit(p301,BIT_SD)} RET={bit(p301,BIT_RET)} TURN={bit(p301,BIT_TURN)}
EM={bit(p301,BIT_EM)} SEL={bit(p301,BIT_SEL)}  DAC=0x{(p301 &
MASK_DAC):03X}")

            self.ind_sel.config(text=f"SEL: {bit(p301, BIT_SEL)} ({'t2' if
bit(p301, BIT_SEL) else 't1'})")
            self.ind_gt.config(text=f"GT: {bit(p300, BIT_GT)}")

            self.ind_em.config(text=f"EM (заслонка): {'ON' if bit(p301, BIT_EM)
else 'OFF'}")
            self.ind_turn.config(text=f"TURN (поворот): {'ON' if bit(p301,
BIT_TURN) else 'OFF'}")
            self.ind_ret.config(text=f"RET (возврат): {'ON' if bit(p301, BIT_RET)
else 'OFF'}")

```

```

        dac_code = p301 & MASK_DAC
        u = dac_u_from_code(dac_code)
        self.ind_dac.config(text=f"DAC: code=0x{dac_code:03X} ({{dac_code}}) →
U≈{u:.2f}V")

        self.root.after(100, self.ui_update)

    def on_close(self):
        self.running = False
        self.root.destroy()

def main():
    root = tk.Tk()
    Variant8Sim(root)
    root.mainloop()

if __name__ == "__main__":
    main()

```

Вывод: изучены принципы организации инфообмена между ядром СРВ и периферийными устройствами, приобретены практические навыки разработки функциональных схем, алгоритмов и управляющих программ для управления процессами в режиме реального времени.